

Python for Embedded/Linux Devices

Service Management (systemd), Process Insight
(ps), Config (Env Vars)

systemd

When your Raspberry Pi boots up, systemd manages the startup process.

systemd is the init system used by Raspberry Pi OS. It starts, stops, and manages services that run automatically in the background.

What does systemd do?

Think of systemd as the conductor of the Linux orchestra:

- Starts services at boot
- Ensures they start in the right order
- Restarts services if they crash
- Allows manual control using commands

Why it's useful for your Python app

If you've written a Python app (like a web API or sensor collector), you can use systemd to:

- Start it automatically at boot
- Keep it running without user login
- Restart it automatically if it fails
- Your `.service` file defines how the system runs your app.

Understanding a .service File

Example: /etc/systemd/system/myapp.service

[Unit]

Description=My Python App Service

After=network.target

[Service]

ExecStart=/usr/bin/python3 /home/pi/myapp/app.py

WorkingDirectory=/home/pi/myapp

Restart=always

User=pi

Environment="PATH=/usr/bin:/usr/local/bin"

Environment="FLASK_ENV=production"

[Install]

WantedBy=multi-user.target

[Unit]

Defines metadata and dependencies.

Description – a short explanation of what the service does (shows up in systemctl status).

After=network.target – ensures the service starts after the network is ready.

[Service]

Defines *how the program actually runs*.

ExecStart – the full command used to launch your program.

WorkingDirectory – the folder the service runs in (important for file paths or virtual environments).

Restart=always – restarts the service automatically if it crashes.

User=pi – the user account that runs the service.

Environment= – optional environment variables

[Install]

Defines when the service should start.

WantedBy=multi-user.target – means it will start automatically during a normal boot (without a desktop GUI).

Common **systemd** commands

Services are managed by systemd's **systemctl** process manager

```
sudo systemctl daemon-reload — reload systemd (pick up new services)
```

```
sudo systemctl start myapp.service — Start now
```

```
sudo systemctl stop myapp.service — Stop
```

```
sudo systemctl status myapp.service — Check status and logs
```

```
sudo systemctl enable myapp.service — Start at boot
```

```
sudo systemctl disable myapp.service — Don't start at boot
```

```
journalctl -u myapp.service — View logs
```

ps (Process Status)

- **ps** shows snapshots of running processes—who they are, what they're doing.
- Common invocations(classic BSD style):

```
ps a          # your shell's processes
ps aux        # list all process including background
ps p 1234      # a specific PID(process ID)
ps aux| grep myservice.py #show processes that contain myservice.py
```

Environment variables: what & why

- Key–value strings provided by the OS to configure apps
- Keep **secrets** & **configs** out of code
- Example use: Change behavior per environment (dev/staging/prod) without code edits
- Common examples: **PATH, HOME, DATABASE_URL, API_KEY**

```
import os
home = os.getenv("HOME")           # Returns None if missing
path = os.getenv("PATH", "")       # Default fallback
api  = os.environ["API_KEY"]       # Raises KeyError if missing (good for required config)
```

The os module (Python 3) at a glance

The os module is Python's cross-platform doorway to the operating system. It abstracts differences between Windows, macOS, and Linux so **your code stays portable**.

- **Filesystem helpers** create, list, move, and delete files/dirs.
 - os.listdir, os.makedirs, os.remove, os.chdir, os.getcwd.
- **Environment access:** read and set process environment via os.environ and os.getenv—handy for config and secrets.
- **Paths** (cross-platform): os.path.join, os.path.expanduser, os.path.abspath.

Reading & setting env vars in Python

- **Read with defaults:** `os.getenv("USER_ID", "fxwalsh")`
- **Required values:** access with `os.environ["NAME"]` and fail fast.
- **Set an env var for current process:**

```
import os, subprocess
os.environ["DEBUG"] = "1"           # affects this process
```